

FINAL REPORT

Project 12 | CSE406 — Cyber Security Sessional

ICMP Blind Connection-Reset and Blind Throughput-Reduction Attack against TCP

SUBMITTED BY

Debashri Roy

2105035

Anisa Binte Asad

2105036

September 19, 2026

Contents

1	Introduction	2
2	Experimental Setup	2
2.1	Initial Validation with Mininet	2
2.1.1	Mininet Topology Diagram	3
2.1.2	Mininet Topology Components	3
2.2	Two-Laptop Topology	3
2.2.1	Two-Laptop Topology Diagram	4
2.2.2	Two-Laptop Topology Components	4
2.3	Three-Device Physical Topology Setup	4
2.3.1	Three Device Topology Diagram	4
2.3.2	Three-Device Topology Components	5
2.4	Kernel Strategy	5
2.5	Attack Mechanics (Expected Message Flow)	5
3	Challenges Faced During Implementation	7
4	Packet and Frame Structure	7
4.1	Spoofed ICMP Packet (both attacks) — general structure	8
4.2	Attack 1 — ICMP Fields Used	8
4.3	Attack 2 — ICMP Fields Used	8
5	Attack 1: Blind Connection-Reset — Results	8
5.1	Steps of the Attack	8
5.2	Expected Outcome	10
5.3	Actual Outcome / Observed Output	10
5.4	Was the Attack Successful?	11
6	Attack 2: Blind Throughput-Reduction — Results	11
6.1	Steps of the Attack	11
6.2	Expected Outcome	12
6.3	Actual Outcome	12
6.4	Was the Attack Successful?	14
7	Defense	14
7.1	Defense 1: TCP Sequence-Number Checking	14
7.1.1	Effectiveness	14
7.2	Defense 2: PMTU Floor	14
7.2.1	Expected Outcome	15
7.2.2	Observed Outcome	15
7.3	Defense Effectiveness Summary	16
8	Discussion	16
9	Conclusion	16

1 Introduction

This report studies two related off-path, blind attacks on TCP connections that exploit the way TCP stacks respond to spoofed ICMP error messages. Both attacks are based on the weakness described in RFC 5927 (*ICMP Attacks against TCP*) [1]. In these attacks, the attacker is not on the path between the two TCP endpoints and cannot observe the connection's actual sequence numbers. Instead, the attacker sends spoofed ICMP error packets to the victim, causing the TCP stack to take an action that can disrupt the connection.

Attack 1 — Blind Connection-Reset via ICMP

Historically, TCP stacks could terminate a connection when they received an ICMP *Destination Unreachable* error (Type 3, Code 2/3/4) that appeared to be related to the connection. An off-path attacker can spoof such a packet by guessing the connection's 4-tuple and including a plausible TCP header, without needing to observe the actual traffic. Modern kernels reduce the risk of this attack by checking whether the TCP sequence number in the ICMP packet falls within the current TCP window before processing the message [1].

Attack 2 — Blind Throughput-Reduction via ICMP

Instead of directly resetting the connection, the attacker sends spoofed *soft errors* that do not terminate the connection but can reduce its performance. In this attack, the attacker spoofs an ICMP *Fragmentation Needed* (Type 3, Code 4) message with a small Next-Hop MTU value, causing the victim to reduce its assumed Path MTU. The reduced PMTU can increase fragmentation and retransmission overhead, resulting in lower effective TCP throughput [1].

2 Experimental Setup

The environment used to run the attacks differs from the setup proposed in the design report. The design report proposed three Linux network namespaces (Client, Server, and Attacker) on a single machine. During the implementation and validation process, we modified the experimental setup multiple times to address practical limitations and validate the attacks under different configurations.

2.1 Initial Validation with Mininet

Before moving to physical hardware, we used Mininet to emulate the intended topology, with Client, Server, and Attacker represented as separate hosts on a single machine. This allowed us to quickly verify that the addressing and traffic flow behaved as expected before deploying the attacks on real hardware.

However, Mininet hosts share the same underlying Linux kernel, which limited our ability to reproduce the intended attack conditions independently for each host. After validating the basic topology and traffic flow in Mininet, we changed the experimental environment to a physical two-laptop topology to better match the requirements of the actual attack demonstration. In this setup, one laptop hosts both the Client and Server endpoints and runs the modified kernel supplied by our supervisor, while a separate laptop acts as an off-path Attacker. This allowed us to demonstrate attacks under the required kernel configuration while keeping the Attacker physically separate from the Client–Server communication path.

2.1.1 Mininet Topology Diagram

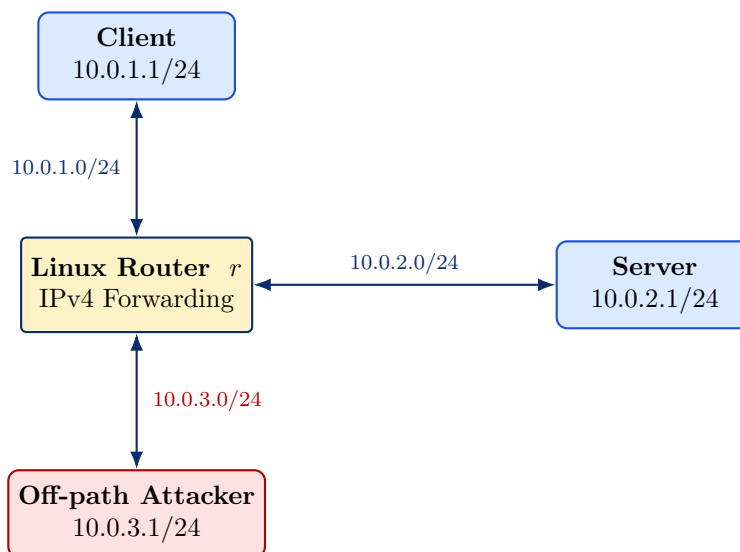


Figure 1: Mininet topology defined in `topo.py`.

2.1.2 Mininet Topology Components

The baseline validation environment is implemented in `topo.py` using Mininet to simulate a multi-subnet network topology. The role and behavior of each node configured within the script are summarized in Table 1.

Node / Role	Description
Client (10.0.1.1/24)	Initiates long-lived TCP connections (e.g., <code>iperf3</code> sessions) to the server across subnet 10.0.1.0/24.
Server (10.0.2.1/24)	Accepts incoming TCP traffic and handles bulk data transfer over subnet 10.0.2.0/24.
Linux Router <i>r</i>	Acts as the central gateway connecting all subnets; enables IPv4 forwarding between Client, Server, and Attacker interfaces.
Off-path Attacker (10.0.3.1/24)	Positioned on subnet 10.0.3.0/24 with no direct access to the Client↔Server path; sends spoofed traffic and infers TCP connection parameters.

Table 1: Roles and network configurations defined in `topo.py`.

2.2 Two-Laptop Topology

- **Laptop 1 — Client + Server:** hosts both TCP endpoints of the victim connection and runs the modified kernel (branch `custom_5.15`) supplied by our supervisor.
- **Laptop 2 — Attacker:** runs independently, off the Client–Server path; it sends only spoofed packets and cannot observe the real Client–Server traffic.

2.2.1 Two-Laptop Topology Diagram

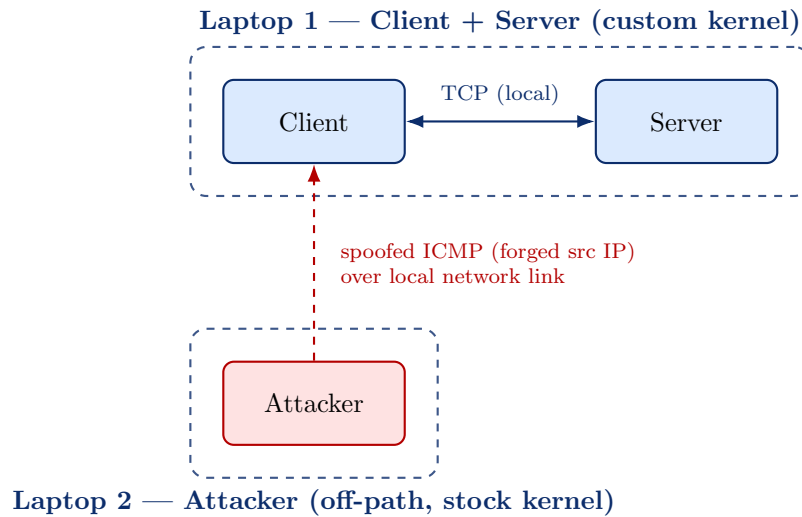


Figure 2: Topology actually used: Client and Server co-located on Laptop 1, Attacker on a physically separate Laptop 2, connected over a local network link

2.2.2 Two-Laptop Topology Components

Role	Description
Laptop 1 – Client (Victim A)	Initiates a long-lived TCP connection (bulk transfer / <code>iperf3</code> session) to the Server on the same laptop.
Laptop 1 – Server (Victim B)	Accepts the TCP connection, sends/receives bulk data; co-located with the Client on the custom-kernel laptop.
Laptop 2 – Off-path Attacker	Has no interception capability on the Client↔Server path; can only send spoofed packets.

Table 2: Two-Laptop Topology Components.

2.3 Three-Device Physical Topology Setup

To evaluate the attack vector in a dedicated physical network environment, a three-device hardware testbed was configured. Unlike the two-laptop setup, this setup isolates the Client, Server, and Off-path Attacker across three distinct physical machines connected via an intermediate router, fully mirroring a real-world enterprise network scenario.

2.3.1 Three Device Topology Diagram

The physical architecture and traffic flow for the three-device deployment are illustrated in Figure 3.

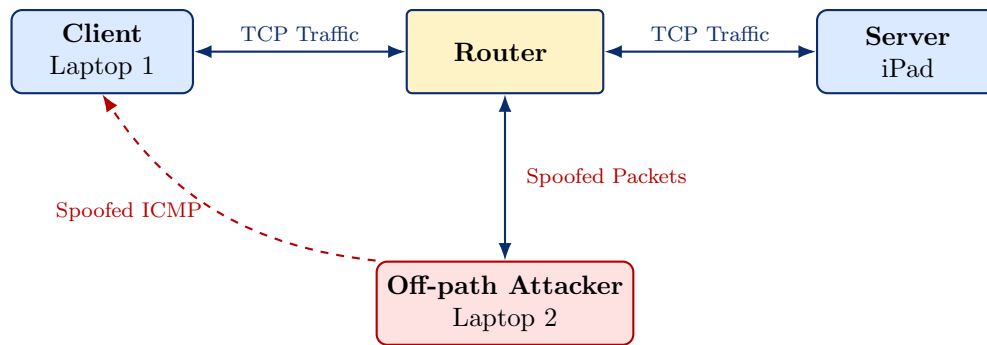


Figure 3: Physical three-device network topology depicting isolated Client, Server, and Attacker connected through a router.

2.3.2 Three-Device Topology Components

The specific roles and system properties of each hardware entity in the three-device environment are detailed in Table 3.

Role / Node	Description
Device 1 – Client	Dedicated physical machine initiating long-lived TCP connections (e.g., <code>iperf3</code> sessions or bulk transfers) to the Server.
Device 2 – Server	Dedicated physical host receiving TCP connections and maintaining active connection state.
Device 3 – Off-path Attacker	Physically isolated attacker connected to the network segment; has no path-interception capability but transmits spoofed ICMP/TCP control packets.
Router	Intermediate network device facilitating packet routing and forwarding between all three endpoints.

Table 3: Component breakdown and operational roles for the three-device physical topology.

2.4 Kernel Strategy

Modern Linux kernels incorporate validation and error-handling mechanisms that mitigate the vulnerability behavior targeted by the attacks [1]. Therefore, the attack demonstrations were conducted using the modified Linux kernel supplied by our supervisor (branch `custom_5.15`), available at `custom_5.15`, with Laptop 1 booted into this kernel. This configuration was necessary to reproduce the vulnerability conditions assumed by the attack scenarios and to evaluate their effects end-to-end under the intended kernel behavior.

2.5 Attack Mechanics (Expected Message Flow)

The diagrams below show the intended message flow for each attack; Sections 5 and 6 report what was actually observed when these were carried out.

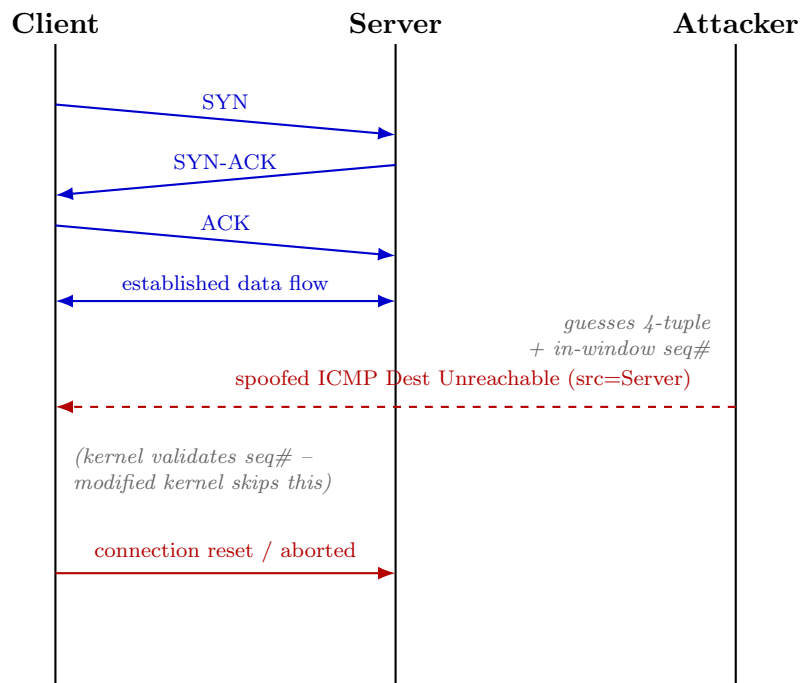
Attack 1 (Blind Connection-Reset)

Figure 4: Sequence diagram for Attack 1 (Blind Reset).

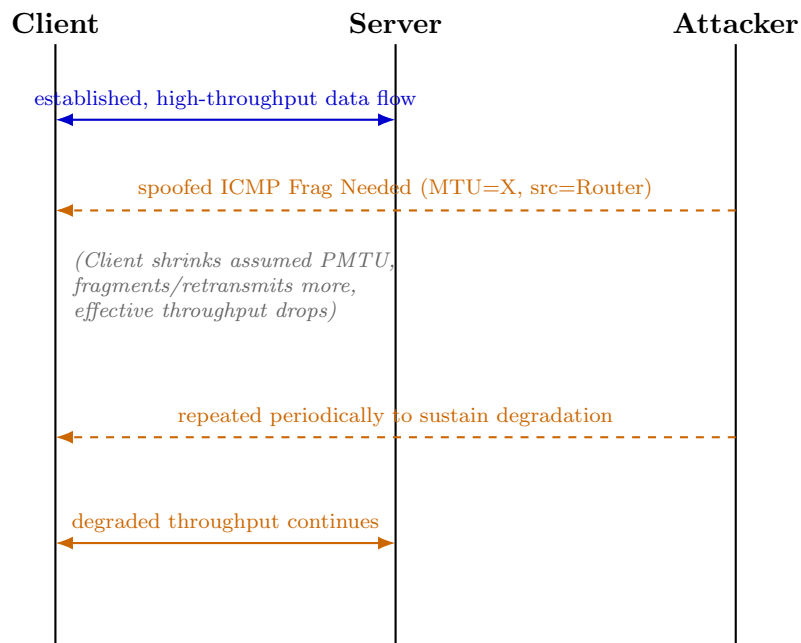
Attack 2 (Blind Throughput-Reduction)

Figure 5: Sequence diagram for Attack 2a (PMTU Reduction).

Implementation Assumptions

- **Known connection parameters:** The Attacker is assumed to know the Client and Server IP addresses and the relevant TCP port. This was necessary because, in the physical three-device Wi-Fi topology, the Attacker could not passively observe the Client–Server unicast traffic.
- **Limited sequence-number guesses:** For Attack 1, the modified kernel used in the experiment relaxes the TCP sequence-number validation. Therefore, only 10 spoofed packets were generated rather than performing an extensive sequence-number sweep, as additional guesses were unnecessary under the experimental kernel configuration.

3 Challenges Faced During Implementation

Several practical challenges were encountered while transitioning from the proposed setup to the final experimental environment:

- **Virtual machine performance:** The experiment was initially planned using VirtualBox. However, the virtualized environment was too slow to handle the custom-kernel setup efficiently, including downloading and working with the kernel source. We therefore moved to the two-laptop topology.
- **Custom-kernel compilation:** The first two kernel-build attempts were unsuccessful and required additional troubleshooting. In particular, an issue involving the `xrealloc()` function had to be resolved before the kernel could be compiled successfully.
- **Network configuration after reboot:** Following the successful installation and reboot into the custom kernel, Wi-Fi connectivity on Laptop 1 was unavailable. Additional network-driver/configuration troubleshooting was required before the experimental setup could be used.
- **TCP sequence-number assumption:** The initial attack design considered sequence-number guesses based on the assumed TCP receive window. During implementation, the experiments were instead conducted under the vulnerability condition in which the relevant TCP sequence-number check in `tcp_v4_err()` is disabled by the custom kernel. This allowed the attack mechanisms to be evaluated without relying on extensive sequence-number guessing.
- **Minimum PMTU assumption:** The initial Attack 2 implementation assumed a minimum supported PMTU of 68 bytes. During kernel-level investigation using `grep` and Bash commands, we identified that the relevant kernel logic enforces a minimum PMTU of 552 bytes. The attack logic was therefore updated to respect this constraint.

4 Packet and Frame Structure

4.1 Spoofed ICMP Packet (both attacks) — general structure

Ethernet Header
IP Header src = spoofed (Server/Router) dst = Victim (Client) proto = 1 (ICMP)
ICMP Header type / code checksum Attack 1 (Blind Reset): Type 3 / Code 3 (Port Unreachable) or Code 1 Attack 2 (Frag-Needed): Type 3 / Code 4 ← carries forged Next-Hop MTU
Embedded (offending) packet Original IP header : src = Client dst = Server proto = 6 (TCP) Original TCP header (8B+) : src port, dst port sequence number ← must be in-window for stack to act (hard errors only)

Figure 6: Generic ICMP packet construction for ICMP-based TCP attacks across topology setups.

4.2 Attack 1 — ICMP Fields Used

- **ICMP Message Type: Type 3** (Destination Unreachable) using **Code 2** (Protocol Unreachable) or **Code 3** (Port Unreachable). Stacks derived from BSD process these as hard errors, triggering an immediate connection abort (ECONNRESET).
- **4-Tuple Matching:** The inner IP+TCP header in the ICMP payload must match the target connection's 4-tuple. The attacker must guess or brute-force the source port if it is not already known.
- **Sequence Number Validation:** Modern patched kernels verify that the embedded TCP sequence number falls within the valid window (SND.UNA to SND.NXT) before acting on the error. In our setup, this validation is relaxed in the modified kernel for demonstration.

4.3 Attack 2 — ICMP Fields Used

- **PMTU Reduction Vector: Type 3** (Destination Unreachable), **Code 4** (Fragmentation Needed and DF set). This packet includes a forged Next-Hop MTU value.
- **Impact of Fake MTU:** Injecting an artificially low MTU causes the sender to reduce its effective MSS. This increases packet overhead and can result in throughput degradation.

5 Attack 1: Blind Connection-Reset — Results

5.1 Steps of the Attack

1. **Establish the Client–Server connection.** On Laptop 1 (Client), the script `run_cs_wifi.sh` was executed to start the TCP connection and keep it active for the experiment.
2. **Start the Server.** On the iPad (Server), the `iPerf Falcon` application was configured to listen on TCP port 5201. Once the connection was established, the Client displayed the

active connection in its terminal, while the iPad showed that the server was ready to receive traffic. This established connection was used as the baseline state for both attacks.

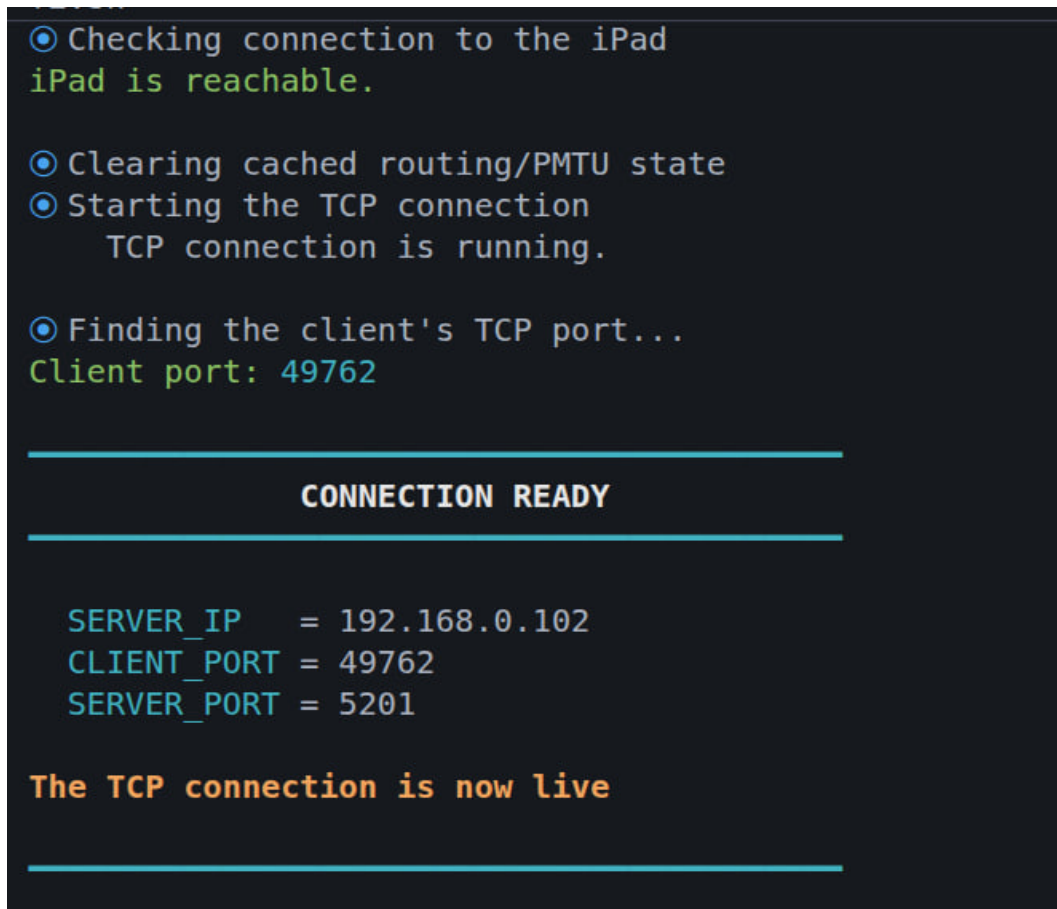


Figure 7: Established Client-side TCP connection before the attack.

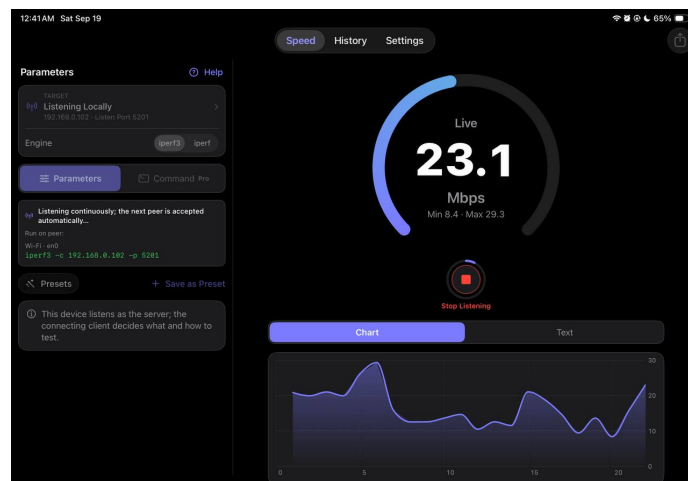


Figure 8: iPerf Falcon on the iPad showing the Server ready for the TCP connection.

3. **Launch the Attacker.** On Laptop 2 (Attacker), the script `run_attack1_wifi.sh` was executed. This script uses `attack1_reset_wifi.sh` to generate the spoofed ICMP Destination Unreachable packets used for the connection-reset attack.

```
◎Attack configuration:
  Client IP   : 192.168.0.108
  Client port : 49770
  iPad IP     : 192.168.0.102
  iPad port   : 5201

◎Sending forged RST packets...

◎Attack configuration:
  Client      : 192.168.0.108
  Server      : 192.168.0.102
  Client port : 49770
  Server port : 5201
◎Route to client:
  interface : wlp1s0
  gateway   : 0.0.0.0
  source    : 192.168.0.114
◎Sending 10 forged ICMP packets...
◎Sent 10 packets in 0.36 seconds.
◎Attack finished - check the client TCP connection state.

◎attack1_reset_wifi.py finished successfully.
```

Figure 9: Attacker-side execution of Attack 1 using `run_attack1_wifi.sh`.

4. **Observe the Client and Server after the attack.** The Client-side kernel output and the Server-side connection state were then inspected to determine whether the spoofed ICMP packets caused the established TCP connection to reset.

5.2 Expected Outcome

- The spoofed ICMP Destination Unreachable packets cause the established TCP connection to be reset.
- The Client terminates the affected TCP connection after processing the forged ICMP error.
- The iPerf Falcon application on the iPad shows a throughput of 0.0 Mbps after the connection is terminated.

5.3 Actual Outcome / Observed Output

Victim PC (Laptop 1). The Client-side kernel output provides evidence that the spoofed ICMP packet was processed as a TCP reset condition. The `dmesg` output shows the corresponding TCP reset message generated by the modified kernel.

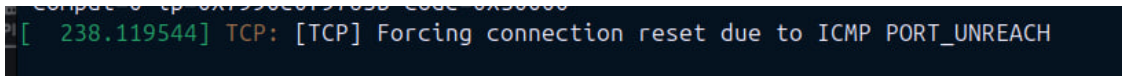


Figure 10: Client-side `dmesg` output showing the TCP connection reset triggered by the spoofed ICMP packet.

Server (iPad). The iPerf Falcon application on the iPad provides evidence that the previously established connection was terminated following the attack.

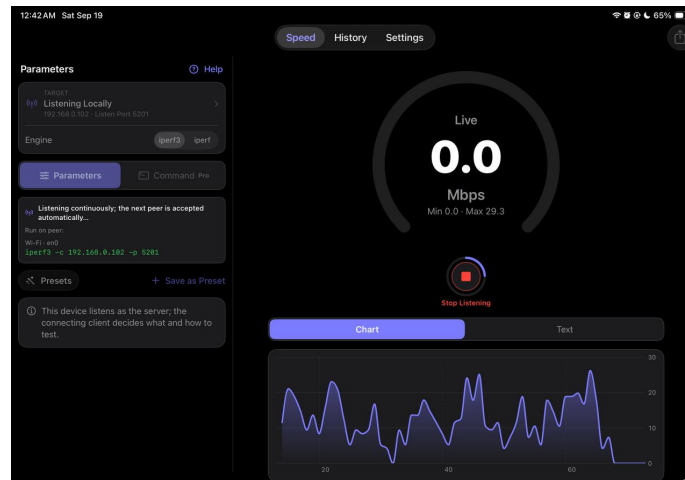


Figure 11: iPerf Falcon showing evidence of the Server-side connection reset after Attack 1.

5.4 Was the Attack Successful?

The attack was successful. The established TCP connection between the Client and Server was terminated after the spoofed ICMP Destination Unreachable packets were sent. The Client-side `dmesg` output confirms that the modified kernel processed the ICMP error as a TCP reset condition, while the iPerf Falcon output on the Server shows that the active connection was no longer transmitting data.

6 Attack 2: Blind Throughput-Reduction — Results

6.1 Steps of the Attack

1. **Establish the Client–Server connection.** The Client–Server connection was established using the same setup described in Section 5. The connection was kept active while the Attacker performed the PMTU reduction attack.
2. **Launch the Attacker.** On Laptop 2 (Attacker), the script `run_attack2_wifi.sh` was executed. This script uses `attack2_pmtu_wifi.py` to generate and send the spoofed ICMP Fragmentation Needed messages containing a reduced Next-Hop MTU value.

```

⊙ Forged Path-MTU: 68 bytes
⊙ Route to client:
  interface : wlp1s0
  gateway   : 0.0.0.0
  source    : 192.168.0.114
⊙ Sending 1651-packet seq-sweep rounds every 2.0s for 40s...
⊙ Round 1: sent 1651 forged PMTU=68B packets.
⊙ Round 2: sent 1651 forged PMTU=68B packets.
⊙ Round 3: sent 1651 forged PMTU=68B packets.
⊙ Round 4: sent 1651 forged PMTU=68B packets.
⊙ Round 5: sent 1651 forged PMTU=68B packets.
⊙ Round 6: sent 1651 forged PMTU=68B packets.
⊙ Round 7: sent 1651 forged PMTU=68B packets.
⊙ Round 8: sent 1651 forged PMTU=68B packets.
⊙ Round 9: sent 1651 forged PMTU=68B packets.
⊙ Round 10: sent 1651 forged PMTU=68B packets.
⊙ Round 11: sent 1651 forged PMTU=68B packets.
⊙ Round 12: sent 1651 forged PMTU=68B packets.
⊙ Sent 19812 packets across 12 rounds over 42.3s.
⊙ Attack finished.
  Check the client terminal for the PMTU/MSS snapshots and throughput reduction %.

```

Figure 12: Attacker-side execution of Attack 2 using `run_attack2_wifi.sh`.

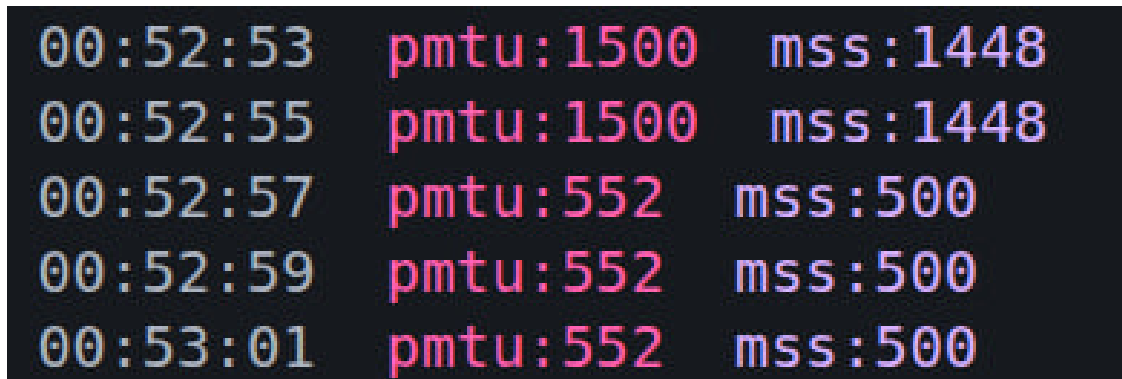
3. **Observe the PMTU and MSS changes.** The Client-side PMTU and TCP MSS values were monitored before and during the attack to determine whether the spoofed ICMP messages caused the effective path MTU to decrease.
4. **Observe the throughput change.** The Client-side `iperf3` throughput was observed at the end to determine whether the PMTU reduction resulted in a measurable decrease in throughput.

6.2 Expected Outcome

- Based on the implementation logic and the minimum PMTU enforced by the experimental kernel, the effective PMTU is expected to decrease from the normal value of 1500 B to 552 B.
- The corresponding TCP MSS is expected to decrease from approximately 1448 B to 500 B.
- The reduced PMTU and MSS increase packet overhead and are expected to result in a reduction in observed throughput.
- The TCP connection remains established; the objective of the attack is to degrade throughput rather than terminate the connection.

6.3 Actual Outcome

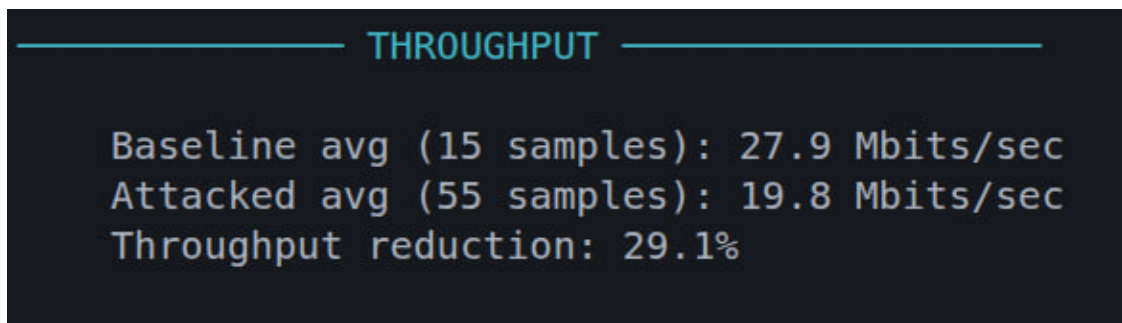
PMTU/MSS Change. The attack successfully modified the effective PMTU observed by the Client. Before the attack, the connection used a PMTU of 1500 B and an MSS of 1448 B. During the attack, these values changed to a PMTU of 552 B and an MSS of 500 B.



```
00:52:53 pmtu:1500 mss:1448
00:52:55 pmtu:1500 mss:1448
00:52:57 pmtu:552 mss:500
00:52:59 pmtu:552 mss:500
00:53:01 pmtu:552 mss:500
```

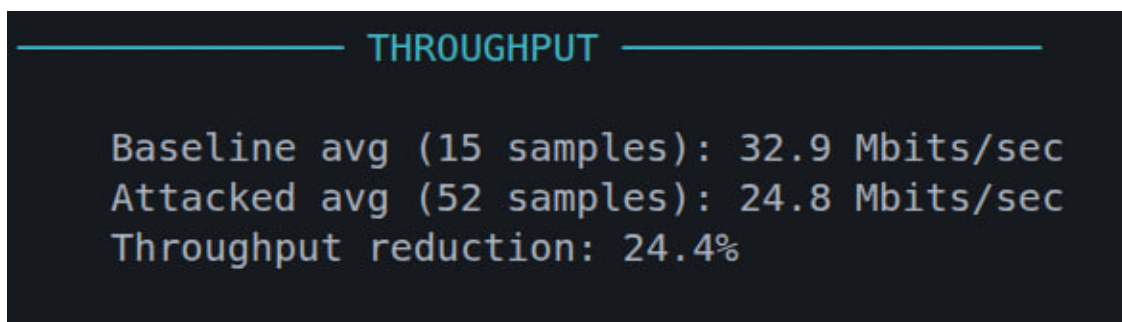
Figure 13: Client-side PMTU and MSS values before and during Attack 2, showing the reduction from 1500/1448 B to 552/500 B.

Throughput Change. The reduction in PMTU and MSS was accompanied by a reduction in observed throughput. The following screenshots show the throughput measurements obtained during the attack.



```
————— THROUGHPUT —————
Baseline avg (15 samples): 27.9 Mbits/sec
Attacked avg (55 samples): 19.8 Mbits/sec
Throughput reduction: 29.1%
```

Figure 14: Observed throughput during an Attack 2 experiment.



```
————— THROUGHPUT —————
Baseline avg (15 samples): 32.9 Mbits/sec
Attacked avg (52 samples): 24.8 Mbits/sec
Throughput reduction: 24.4%
```

Figure 15: Observed throughput during another Attack 2 experiment.

The exact percentage of throughput reduction varies between runs because the baseline throughput is calculated from the initial period of the experiment. Since the experiments were conducted over a real Wi-Fi connection, the baseline throughput was not perfectly constant and naturally fluctuated between runs. Consequently, the measured percentage reduction also varied across experiments, even though the PMTU reduction from 1500 B to 552 B and the corresponding MSS reduction from 1448 B to 500 B were consistently observed.

6.4 Was the Attack Successful?

The attack was successful. The spoofed ICMP Fragmentation Needed messages caused the effective PMTU on the Client to decrease from 1500 B to 552 B, with the corresponding MSS decreasing from 1448 B to 500 B. This reduction increased packet overhead and resulted in a measurable decrease in observed throughput. Although the exact percentage of throughput reduction fluctuated between runs due to the variable nature of the Wi-Fi baseline, the intended PMTU reduction and resulting throughput degradation were successfully reproduced.

7 Defense

7.1 Defense 1: TCP Sequence-Number Checking

Modern Linux kernels use TCP sequence-number validation when processing ICMP errors associated with an established TCP connection. This validation prevents forged ICMP packets with invalid sequence information from being accepted as valid errors for the connection.

To evaluate this defense, the experimental roles were reversed from the attack setup. The Client-side code was executed on Laptop 2, while the Attacker code was executed from Laptop 1 using the stock Linux kernel with the TCP sequence-number checking logic enabled. The same attack procedures were then attempted against the established connection.

Both Attack 1 and Attack 2 failed under this configuration. The forged ICMP packets did not produce the intended connection reset or PMTU reduction, demonstrating the protective effect of the TCP sequence-number validation present in the modern kernel.

7.1.1 Effectiveness

- **Attack 1:** The spoofed ICMP Destination Unreachable packet did not reset the established TCP connection.
- **Attack 2:** The spoofed ICMP Fragmentation Needed packet did not produce the intended PMTU reduction or corresponding throughput degradation. The defense was tested using TCP sequence-number validation with an assumed receive-window range. However, this approach is not practical for a blind attacker because the initial TCP sequence number can take any value in the 32-bit sequence-number space (0 to $2^{32} - 1$). Brute-forcing the entire sequence-number space is therefore infeasible, making successful blind injection highly impractical.

7.2 Defense 2: PMTU Floor

For Attack 2, an additional defense was implemented by increasing the minimum accepted PMTU on the Client. The Linux kernel parameter `net.ipv4.route.min_pmtu` was used to impose a lower bound on the PMTU accepted through Path MTU Discovery.

The default PMTU floor observed in the experimental kernel was 552 B. The defense script temporarily increased this value to 1200 B before starting the Client–Server experiment. As a result, spoofed ICMP Fragmentation Needed messages specifying an MTU below 1200 B could not reduce the effective PMTU below the configured floor.

The defense was implemented using a wrapper script, `run_cs_wifi_defended.sh`, which:

1. Reads and stores the original value of `net.ipv4.route.min_pmtu`.
2. Sets the PMTU floor to 1200 B using `sysctl`.

3. Starts the normal Client–Server experiment using `run_cs_wifi.sh`.
4. Restores the original PMTU floor when the experiment terminates, including when interrupted.

```
=====
PMTU FLOOR DEFENSE ENABLED
=====

Current PMTU floor : 552
New PMTU floor      : 1200
```

Figure 16: PMTU floor defense enabled on the Client, with the minimum PMTU configured to 1200 B.

7.2.1 Expected Outcome

- The Client accepts a minimum PMTU of 1200 B rather than the default 552 B.
- A spoofed ICMP Fragmentation Needed message attempting to reduce the PMTU below 1200 B should not cause the PMTU to fall below this configured floor.
- Consequently, the severe PMTU reduction observed in Attack 2 should be prevented.

7.2.2 Observed Outcome

With the PMTU floor defense enabled, the Client did not accept the attempted PMTU reduction below the configured minimum. The observed PMTU remained bounded by the configured floor rather than reaching the 552 B value observed without the defense.

```
01:42:53 pmtu:1500 mss:1448
01:42:55 pmtu:1500 mss:1448
01:42:57 pmtu:1200 mss:1148
01:42:59 pmtu:1200 mss:1148
```

Figure 17: Observed PMTU behavior with the 1200 B minimum PMTU defense enabled.

The result shows that increasing `net.ipv4.route.min_pmtu` limits the extent to which a spoofed ICMP Fragmentation Needed message can reduce the effective PMTU. This provides a practical mitigation for the PMTU-reduction attack, although it does not eliminate the underlying reliance on unauthenticated ICMP messages.

7.3 Defense Effectiveness Summary

Defense	Expected Effect	Observed Effect
Defense 1: TCP sequence-number checking	Forged ICMP packets with invalid TCP sequence information are rejected	Both Attack 1 and Attack 2 failed under the stock kernel
Defense 2: PMTU floor	PMTU cannot be reduced below the configured 1200 B floor	The attempted PMTU reduction was bounded by the configured floor

Table 4: Summary of the implemented defenses and their observed effects.

8 Discussion

- **Effect of kernel validation.** The stock kernel's TCP sequence-number validation prevented the spoofed ICMP packets from producing the intended attack effects, while the modified kernel allowed the vulnerability behavior to be reproduced.
- **Challenge of blind attacks.** Since the Attacker cannot observe the Client–Server traffic, the TCP sequence number must be guessed. Although an assumed receive-window range can be tested, the initial sequence number can be anywhere in the 32-bit space (0 to $2^{32} - 1$), making brute-force guessing impractical.
- **Effect of PMTU reduction.** Attack 2 demonstrated that spoofed ICMP messages can reduce PMTU and MSS, increasing packet overhead and degrading throughput without terminating the TCP connection.
- **Effect of the PMTU floor.** Increasing `net.ipv4.route.min_pmtu` to 1200 B limited the PMTU reduction and prevented the severe reduction to 552 B observed without the defense.
- **Experimental variability.** The exact throughput reduction varied between runs because the experiments were conducted over real Wi-Fi, where baseline throughput naturally fluctuates. The PMTU/MSS changes were therefore a more consistent indicator of the attack.
- **Kernel-level investigation.** The experiments showed the importance of verifying kernel behavior rather than relying on assumptions. The initial 68 B PMTU assumption was corrected after kernel investigation revealed a 552 B minimum.
- **Overall observation.** Attack 1 demonstrated direct TCP connection disruption, while Attack 2 demonstrated performance degradation. The experiments also showed that kernel validation and PMTU-floor configuration can significantly limit these attack effects.

9 Conclusion

This project was a challenging but valuable learning opportunity. Through implementing and testing the two ICMP-based attacks, we gained a deeper practical understanding of TCP, ICMP, PMTU and Linux networking. It also helped us understand Computer Networks much better than we did through coursework alone last semester.

We sincerely thank our supervisor for providing the custom Linux kernel, which was essential for reproducing the intended vulnerability behavior. Without the custom kernel and our supervisor's guidance, we would likely have been stuck for a much longer time.

References

1. RFC 5927 — ICMP Attacks against TCP
2. RFC 4821 — Packetization Layer Path MTU Discovery
3. Reference kernel: `custom.5.15` branch